

BidForge AI

AI-Powered RFP Analysis & Proposal Generation Platform

Technical & User Documentation

Version 1.0 · 12 June 2026

Contents

1. Overview
2. System Architecture
3. Getting Started — Installation & Setup
4. The Bid Pipeline (How a Bid Flows Through the System)
5. Application Pages (User Guide)
6. Document Exports
7. API Reference
8. Capability Library
9. LLM Strategy — Caching, Fallback & Demo Mode
10. Database Schema & Migrations
11. Troubleshooting & Known Behaviours

1. Overview

BidForge AI is an end-to-end bid management platform for companies that respond to RFPs (Requests for Proposal). It ingests an RFP document, extracts every requirement automatically, checks each one against the company's documented capability library, drafts proposal narrative with AI, computes a win-probability score with a GO/NO-GO recommendation, and exports a formatted proposal as a Microsoft Word document.

The platform answers four questions for every incoming RFP:

- What exactly is the client asking for? (Parse — requirement extraction)
- Can we prove we have done this before? (Match — RAG compliance matching)
- What should our response say? (Draft — AI proposal generation with evidence citations)
- Should we bid at all? (Score — win-probability model trained on historical bid outcomes)

2. System Architecture

Layer	Technology	Role
Frontend	React 18 + Vite + Tailwind CSS	Single-page app: dashboard, workspace, proposal editor, library management
Backend	FastAPI (Python 3.12) + Uvicorn	REST API + Server-Sent Events streaming, all pipeline logic
Database	Supabase (PostgreSQL)	Workspaces, requirements, compliance items, proposal sections, bid scores
Vector search	FAISS + fastembed (all-MiniLM-L6-v2, ONNX)	Dense retrieval over the capability library; hybrid re-ranking with metadata
LLM (primary)	GROQ — llama-3.3-70b-versatile	Requirement extraction, compliance matching, proposal drafting, summaries
LLM (fallback)	OpenRouter free-model rotation	Automatic failover when GROQ is rate-limited or down
Win model	scikit-learn LogisticRegression	Win probability from 6 factors, trained on data/bid_history.csv
Document output	python-docx	Draft export and full 8-section proposal export

Project layout

BidForge AI/
app/

main.py
routers/
services/

models/schemas.py
utils/
data/
frontend/
scripts/
schema.sql
venv/
.env

FastAPI app, startup, /health
workspaces, bid_history, capabilities, settings
document_parser, requirement_extractor,
compliance_matcher, proposal_generator,
scoring_engine, win_model, docx_exporter,
capability_store, llm_fallback ...
Pydantic request/response models
supabase_client, llm_cache
capability_library(.enriched).json, cache/
React app (Vite)
warm_cache, enrich_capabilities, utilities
Supabase DDL + migration notes
Python virtual environment
API keys & runtime flags

3. Getting Started — Installation & Setup

3.1 Prerequisites

- Python 3.12 (the project venv lives at `.\venv` — there is no system Python on the dev machine)
- Node.js 18+ (frontend)
- A Supabase project (run `schema.sql` in its SQL editor)
- A GROQ API key (free tier works; 100k tokens/day cap)
- Optionally an OpenRouter API key for automatic failover

3.2 Environment variables (.env)

Variable	Purpose
SUPABASE_URL / SUPABASE_KEY	Supabase project connection
GROQ_API_KEY	Primary LLM provider
OPENROUTER_API_KEY	Fallback LLM rotation (optional but recommended)
DEMO_MODE	true = serve nearest cached LLM response when all providers fail (judge-day insurance)
MANUAL_BASELINE_HOURS	Manual-effort baseline used for the time-saved metric

3.3 Running the backend

```
.\venv\Scripts\python.exe -m uvicorn app.main:app --host 0.0.0.0 --port 8000
```

Interactive API docs are served at `http://localhost:8000/docs`; liveness at `/health`.

3.4 Running the frontend

```
cd frontend
npm install
npm run dev      # development, proxies /api to :8000
npm run build    # production bundle in frontend/dist
```

4. The Bid Pipeline

Every bid lives in a workspace and moves through six stages. Each stage is one API call, triggered from the UI; the workspace status field tracks progress (created → uploaded → parsed → matched → drafted → scored → exported).

Stage	Endpoint	What happens
1. Upload	POST /api/workspaces	PDF/DOCX RFP is uploaded and stored; a workspace row is created.
2. Parse	POST /api/workspaces/{id}/parse	Document text is chunked and sent to the LLM (plus an NER pass); structured requirements are extracted with section refs, types (mandatory / question / evaluation criteria / submission deadline / budget), deadlines, and budget references. Sector and issuer type are inferred.
3. Match	POST /api/workspaces/{id}/match	Each requirement is matched against the capability library with hybrid retrieval (50% dense cosine + 50% structured metadata: domain, certification, recency, client type). Produces PASS / PARTIAL / FAIL compliance items with confidence and gap notes.
4. Draft	POST /api/workspaces/{id}/draft	For every mandatory/question requirement, the top-3 matching capabilities are fed to the LLM, which writes a 2-3 paragraph evidence-grounded response. Sections stream token-by-token to the editor (SSE) and are batch-saved to the database.
5. Score	POST /api/workspaces/{id}/score	Six factors (compliance rate, domain match, budget alignment, past win rate, capability depth, competitor presence) feed a logistic-regression win model → overall score and GO /

		CONDITIONAL / NO-GO decision.
6. Export	POST /api/workspaces/{id}/export or /export-full	Generates the proposal DOCX (see section 6).

5. Application Pages (User Guide)

5.1 Dashboard

- Stat cards: total bids, GO decisions, average win score, proposals exported, average time saved vs manual.
- Status filter chips: All / Pending / In Progress / Done.
- One card per bid showing sector, status, compliance bar, win score, and decision badge.
- Analyze New RFP opens the upload modal.
- Delete: hover a card and click the trash icon to permanently delete the bid and all of its data (a confirmation prompt is shown first).

5.2 Active Bids

- Table view of every workspace with search across name, sector, status, and decision.
- Click a row to open the workspace; the trash icon on each row deletes the bid.

5.3 Workspace (master-detail)

- Left column: every extracted requirement with section ref, type badge, and filter tabs (All / Mandatory / Evaluation / Deadline).
- Centre column: the selected requirement, its compliance analysis (status, matched capability, confidence bar, gap note), and the drafted proposal content for it.
- Right column: win-score gauge, GO/NO-GO badge, score breakdown bars, competitor-presence input (re-scores live), and top gaps.
- My Note: each requirement has a short note box — type a note and press Save note. Notes are saved to the database and shown as a small preview in the requirement list.
- Mark as done: a per-requirement toggle. Done requirements show a green check and strikethrough in the list, so you can track review progress through a long RFP.

5.4 Proposal Editor

- Generate Proposal Draft streams one AI-written section per requirement, live, token by token.
- Each section can be edited (Edit → Save Draft) and approved (Approve). Edits and approvals are saved to the database immediately; the approved state persists across refreshes.
- If you approve while a freshly generated draft is still being saved, the editor tells you to retry in a few seconds instead of faking success.
- Two export buttons (see section 6): Export Full Proposal and Export Draft DOCX. Buttons show progress and real error messages; exports can be repeated any number of times.

5.5 Bid History

- Shows the historical bid outcomes dataset (the win model's training data).
- Upload a replacement CSV to retrain the win model.

5.6 Capability Library

- Browse all capability records (past projects) the matcher uses as evidence.
- Upload a replacement JSON library; the FAISS index is rebuilt automatically.

5.7 Settings

- System status (providers, cache statistics) and runtime toggles persisted to .env.
- Clear LLM cache button.

6. Document Exports

Two DOCX exports are available from the Proposal Editor:

6.1 Export Draft DOCX (POST /api/workspaces/{id}/export)

A working draft: cover page, AI executive summary, the proposal narrative (every section in natural order, edited text preferred over raw AI draft), a compliance checklist table, and the win-score analysis page.

6.2 Export Full Proposal (POST /api/workspaces/{id}/export-full)

A complete, submission-shaped proposal with eight parts:

#	Section	Source
1	Cover Page	Company name, RFP reference (workspace name), date
2	Executive Summary	LLM-generated from the drafted sections
3	Understanding of Requirements	LLM-written from the extracted requirements; deterministic fallback if providers fail
4	Technical Proposal	One subsection per requirement: requirement text, your approved/edited draft (preferred over the raw AI draft), and a supporting-evidence citation. Requirements with no matching capability are flagged: [GAP FLAGGED - No evidence found for this requirement]
5	Company Profile & Credentials	Aggregated deterministically from the capability library (practice areas, certifications, project counts) — no LLM, nothing can be fabricated
6	Relevant Past Projects	Three case studies of the capabilities most frequently matched against this RFP, written strictly from library facts
7	Compliance Matrix	Table: Requirement Compliant (Yes/Partial/No) Evidence
8	Appendices	Placeholder list for registration documents,

Every export is written to a unique timestamped file, so repeated exports never collide with files that are still open. All LLM-written parts go through the cache and provider rotation — the first full export of a workspace takes about 1-2 minutes; repeats take seconds.

7. API Reference

Base URL: <http://localhost:8000> — interactive docs at </docs>.

Method	Path	Summary
GET	/health	Liveness probe: service status, capability count, win-model state, demo mode
POST	/api/workspaces	Create a workspace and upload the RFP document
GET	/api/workspaces	List all workspaces
GET	/api/workspaces/dashboard	Bulk dashboard summary for all workspaces
GET	/api/workspaces/{id}	Get one workspace
PATCH	/api/workspaces/{id}	Update manual inputs (competitor presence)
DELETE	/api/workspaces/{id}	Delete a bid and all its data (requirements, compliance, sections, scores, files)
GET	/api/workspaces/{id}/overview	Workspace + requirements + compliance + sections + latest score
POST	/api/workspaces/{id}/parse	Extract structured requirements from the RFP
POST	/api/workspaces/{id}/match	RAG compliance matching against the capability library
POST	/api/workspaces/{id}/draft	Stream AI proposal sections (Server-Sent Events)
POST	/api/workspaces/{id}/score	Win-probability score and GO/NO-GO decision
PATCH	/api/workspaces/{id}/proposal/{section_id}	Save human edits / approval on a proposal section
PATCH	/api/workspaces/{id}/requirements/{req_id}	Save a note and/or done-mark on a requirement
POST	/api/workspaces/{id}/export	Generate and download the draft proposal DOCX
POST	/api/workspaces/{id}/export-full	Generate and

		download the complete 8-section proposal DOCX
GET	/api/bid-history	Get the historical bid outcomes dataset
POST	/api/bid-history/upload	Replace bid history CSV and retrain the win model
GET	/api/capabilities	Get the capability library
POST	/api/capabilities/upload	Replace the capability library and rebuild the index
GET	/api/settings	System status and configuration
PATCH	/api/settings	Update runtime configuration (persisted to .env)
POST	/api/settings/clear-cache	Delete all cached LLM responses
GET	/api/debug/match?q=...	Ad-hoc hybrid match against the library
GET	/api/debug/cache-stats	LLM cache hit/miss/fallback statistics

8. Capability Library

The library is the single source of evidence for compliance matching and proposal drafting: 60 past-project records in `app/data/capability_library.json` (an enriched variant with richer summaries is preferred when present). Each record has: `id`, `domain`, `project_title`, `summary`, `certification`, `year_completed`, `contract_value`, `duration_months`, `client_type`.

Domains (5-10 projects each)

- Engineering & infrastructure: Road Construction, Bridge Engineering, Network Design, Solar Energy, Fleet Management, Medical Equipment
- IT & software: Cybersecurity, Hospital IT, ERP Implementation, LMS Development, Custom Software Development, Mobile App Development, Cloud Migration, DevOps & Automation, Data Analytics & BI, AI & Machine Learning, E-Commerce Platforms, Fintech Solutions, SaaS Product Engineering, IT Managed Services

Hybrid retrieval

Top-10 candidates are retrieved by dense cosine similarity (fastembed all-MiniLM-L6-v2 vectors in a FAISS inner-product index), then re-ranked by hybrid score = $0.5 \times \text{dense} + 0.5 \times \text{structured}$, where the structured score weighs domain match (45%), certification match (30%), recency (15%), and client-type match (10%).

9. LLM Strategy — Caching, Fallback & Demo Mode

- Cache-first: every LLM call checks a file cache (app/data/cache) keyed by sha256(model + normalized prompt). Hits return instantly; streamed sections replay token-by-token so the typing effect is preserved.
- Provider rotation: when GROQ fails (e.g. the 100k tokens/day cap), the call automatically rotates through configured free OpenRouter models until one answers. The response is cached as usual.
- DEMO_MODE=true: if every provider fails, the nearest cached response with the same call-site tag is served (similarity-gated) — degraded but never empty.
- Deterministic fallbacks: export narrative sections (understanding, case studies, executive summary) degrade to honest, fact-based text rather than failing the export.
- scripts/warm_cache.py pre-populates the cache for a known demo document.

10. Database Schema & Migrations

Six tables in Supabase (full DDL in schema.sql). Children cascade on workspace delete.

Table	Key columns
workspaces	id, name, status, rfp_file_path, sector, issuer_type, competitor_presence, pipeline_timings (JSONB), created_at
requirements	id, workspace_id, section_ref, requirement_type, deadline, budget_ref, extraction_source, taxonomy_id/name, note, is_done
compliance_items	id, requirement_id, workspace_id, status (pass/partial/fail), matched_capability_id, confidence, gap_note
proposal_sections	id, workspace_id, requirement_id, section_title, section_ref, ai_draft, edited_draft, status (draft/edited/approved)
bid_scores	id, workspace_id, six factor columns, overall_score, decision, score_breakdown (JSONB)

Pending migration (run once in the Supabase SQL editor)

```
ALTER TABLE requirements ADD COLUMN IF NOT EXISTS note TEXT;  
ALTER TABLE requirements ADD COLUMN IF NOT EXISTS is_done BOOLEAN DEFAULT FALSE;
```

Until these run, saving a requirement note returns an error message containing exactly this SQL. All other migration notes are kept inline in schema.sql.

11. Troubleshooting & Known Behaviours

Symptom	Cause & fix
Export button does nothing	Check the toast — it now shows the real server error. If the API is unreachable, confirm the backend is running on port 8000 and that no stale Python process is holding the port (Get-NetTCPConnection - LocalPort 8000 -State Listen).
GROQ rate-limit errors in logs	The free tier caps at 100k tokens/day. Calls rotate to OpenRouter automatically; cached responses keep working. Consider a second GROQ key for demos.
Approve/edit seems to fail on old workspaces	Sections saved before the proposal_sections migration have NULL requirement_id / section_title. Saving works; regenerate the draft to restore exact per-requirement evidence citations in the full export (legacy sections fall back to section-group citations).
Saving a requirement note returns a database error	Run the two ALTER TABLE statements from section 10 in the Supabase SQL editor.
Full export is slow the first time	It makes several LLM calls (summary, understanding, 3 case studies). They are cached — repeat exports of the same workspace take a few seconds.
Win score looks wrong after manual edits	Re-run POST /score (the competitor-presence dropdown does this automatically).